
Working with Key/Value Pairs

This chapter covers how to work with RDDs of key/value pairs, which are a common data type required for many operations in Spark. Key/value RDDs are commonly used to perform aggregations, and often we will do some initial ETL (extract, transform, and load) to get our data into a key/value format. Key/value RDDs expose new operations (e.g., counting up reviews for each product, grouping together data with the same key, and grouping together two different RDDs).

We also discuss an advanced feature that lets users control the layout of pair RDDs across nodes: *partitioning*. Using controllable partitioning, applications can sometimes greatly reduce communication costs by ensuring that data will be accessed together and will be on the same node. This can provide significant speedups. We illustrate partitioning using the PageRank algorithm as an example. Choosing the right partitioning for a distributed dataset is similar to choosing the right data structure for a local one—in both cases, data layout can greatly affect performance.

Motivation

Spark provides special operations on RDDs containing key/value pairs. These RDDs are called pair RDDs. Pair RDDs are a useful building block in many programs, as they expose operations that allow you to act on each key in parallel or regroup data across the network. For example, pair RDDs have a `reduceByKey()` method that can aggregate data separately for each key, and a `join()` method that can merge two RDDs together by grouping elements with the same key. It is common to extract fields from an RDD (representing, for instance, an event time, customer ID, or other identifier) and use those fields as keys in pair RDD operations.

Creating Pair RDDs

There are a number of ways to get pair RDDs in Spark. Many formats we explore loading from in [Chapter 5](#) will directly return pair RDDs for their key/value data. In other cases we have a regular RDD that we want to turn into a pair RDD. We can do this by running a `map()` function that returns key/value pairs. To illustrate, we show code that starts with an RDD of lines of text and keys the data by the first word in each line.

The way to build key-value RDDs differs by language. In Python, for the functions on keyed data to work we need to return an RDD composed of tuples (see [Example 4-1](#)).

Example 4-1. Creating a pair RDD using the first word as the key in Python

```
pairs = lines.map(lambda x: (x.split(" ")[0], x))
```

In Scala, for the functions on keyed data to be available, we also need to return tuples (see [Example 4-2](#)). An implicit conversion on RDDs of tuples exists to provide the additional key/value functions.

Example 4-2. Creating a pair RDD using the first word as the key in Scala

```
val pairs = lines.map(x => (x.split(" ")[0], x))
```

Java doesn't have a built-in tuple type, so Spark's Java API has users create tuples using the `scala.Tuple2` class. This class is very simple: Java users can construct a new tuple by writing `new Tuple2(elem1, elem2)` and can then access its elements with the `._1()` and `._2()` methods.

Java users also need to call special versions of Spark's functions when creating pair RDDs. For instance, the `mapToPair()` function should be used in place of the basic `map()` function. This is discussed in more detail in [“Java” on page 43](#), but let's look at a simple case in [Example 4-3](#).

Example 4-3. Creating a pair RDD using the first word as the key in Java

```
PairFunction<String, String, String> keyData =  
    new PairFunction<String, String, String>() {  
        public Tuple2<String, String> call(String x) {  
            return new Tuple2(x.split(" ")[0], x);  
        }  
    };  
JavaPairRDD<String, String> pairs = lines.mapToPair(keyData);
```

When creating a pair RDD from an in-memory collection in Scala and Python, we only need to call `SparkContext.parallelize()` on a collection of pairs. To create a

pair RDD in Java from an in-memory collection, we instead use `SparkContext.parallelizePairs()`.

Transformations on Pair RDDs

Pair RDDs are allowed to use all the transformations available to standard RDDs. The same rules apply from “[Passing Functions to Spark](#)” on page 30. Since pair RDDs contain tuples, we need to pass functions that operate on tuples rather than on individual elements. Tables 4-1 and 4-2 summarize transformations on pair RDDs, and we will dive into the transformations in detail later in the chapter.

Table 4-1. Transformations on one pair RDD (example: `{(1, 2), (3, 4), (3, 6)}`)

Function name	Purpose	Example	Result
<code>reduceByKey(func)</code>	Combine values with the same key.	<code>rdd.reduceByKey((x, y) => x + y)</code>	<code>{(1, 2), (3, 10)}</code>
<code>groupByKey()</code>	Group values with the same key.	<code>rdd.groupByKey()</code>	<code>{(1, [2]), (3, [4, 6])}</code>
<code>combineByKey(createCombiner, mergeValue, mergeCombiners, partitioner)</code>	Combine values with the same key using a different result type.	See Examples 4-12 through 4-14.	
<code>mapValues(func)</code>	Apply a function to each value of a pair RDD without changing the key.	<code>rdd.mapValues(x => x+1)</code>	<code>{(1, 3), (3, 5), (3, 7)}</code>
<code>flatMapValues(func)</code>	Apply a function that returns an iterator to each value of a pair RDD, and for each element returned, produce a key/value entry with the old key. Often used for tokenization.	<code>rdd.flatMapValues(x => (x to 5))</code>	<code>{(1, 2), (1, 3), (1, 4), (1, 5), (3, 4), (3, 5)}</code>
<code>keys()</code>	Return an RDD of just the keys.	<code>rdd.keys()</code>	<code>{1, 3, 3}</code>

Function name	Purpose	Example	Result
<code>values()</code>	Return an RDD of just the values.	<code>rdd.values()</code>	{2, 4, 6}
<code>sortByKey()</code>	Return an RDD sorted by the key.	<code>rdd.sortByKey()</code>	{(1, 2), (3, 4), (3, 6)}

Table 4-2. Transformations on two pair RDDs ($rdd = \{(1, 2), (3, 4), (3, 6)\}$ $other = \{(3, 9)\}$)

Function name	Purpose	Example	Result
<code>subtractByKey</code>	Remove elements with a key present in the other RDD.	<code>rdd.subtractByKey(other)</code>	{(1, 2)}
<code>join</code>	Perform an inner join between two RDDs.	<code>rdd.join(other)</code>	{(3, (4, 9)), (3, (6, 9))}
<code>rightOuterJoin</code>	Perform a join between two RDDs where the key must be present in the first RDD.	<code>rdd.rightOuterJoin(other)</code>	{(3, (Some(4), 9)), (3, (Some(6), 9))}
<code>leftOuterJoin</code>	Perform a join between two RDDs where the key must be present in the other RDD.	<code>rdd.leftOuterJoin(other)</code>	{(1, (2, None)), (3, (4, Some(9))), (3, (6, Some(9)))}
<code>cogroup</code>	Group data from both RDDs sharing the same key.	<code>rdd.cogroup(other)</code>	{(1, ([2], [])), (3, ([4, 6], [9]))}

We discuss each of these families of pair RDD functions in more detail in the upcoming sections.

Pair RDDs are also still RDDs (of `Tuple2` objects in Java/Scala or of Python tuples), and thus support the same functions as RDDs. For instance, we can take our pair RDD from the previous section and filter out lines longer than 20 characters, as shown in Examples 4-4 through 4-6 and Figure 4-1.

Example 4-4. Simple filter on second element in Python

```
result = pairs.filter(lambda keyValue: len(keyValue[1]) < 20)
```

Example 4-5. Simple filter on second element in Scala

```
pairs.filter{case (key, value) => value.length < 20}
```

Example 4-6. Simple filter on second element in Java

```
Function    new Function        public Boolean call(Tuple2<String, String> keyValue) {  
            return (keyValue._2().length() < 20);  
        }  
    };  
JavaPairRDD<String, String> result = pairs.filter(longWordFilter);
```

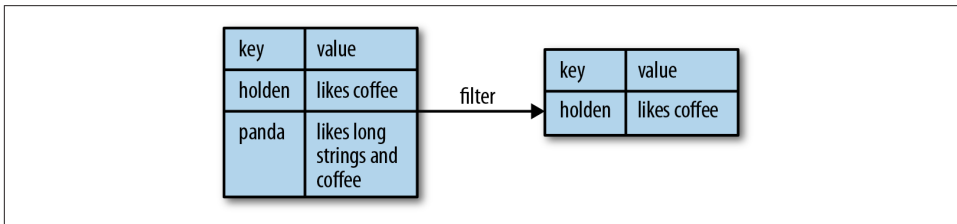


Figure 4-1. Filter on value

Sometimes working with pairs can be awkward if we want to access only the value part of our pair RDD. Since this is a common pattern, Spark provides the `mapValues(func)` function, which is the same as `map{case (x, y): (x, func(y))}`. We will use this function in many of our examples.

We now discuss each of the families of pair RDD functions, starting with aggregations.

Aggregations

When datasets are described in terms of key/value pairs, it is common to want to aggregate statistics across all elements with the same key. We have looked at the `fold()`, `combine()`, and `reduce()` actions on basic RDDs, and similar per-key transformations exist on pair RDDs. Spark has a similar set of operations that combines values that have the same key. These operations return RDDs and thus are transformations rather than actions.

`reduceByKey()` is quite similar to `reduce()`; both take a function and use it to combine values. `reduceByKey()` runs several parallel reduce operations, one for each key in the dataset, where each operation combines values that have the same key. Because datasets can have very large numbers of keys, `reduceByKey()` is not implemented as an action that returns a value to the user program. Instead, it returns a new RDD consisting of each key and the reduced value for that key.

`foldByKey()` is quite similar to `fold()`; both use a zero value of the same type of the data in our RDD and combination function. As with `fold()`, the provided zero value

for `foldByKey()` should have no impact when added with your combination function to another element.

As Examples 4-7 and 4-8 demonstrate, we can use `reduceByKey()` along with `mapValues()` to compute the per-key average in a very similar manner to how `fold()` and `map()` can be used to compute the entire RDD average (see Figure 4-2). As with averaging, we can achieve the same result using a more specialized function, which we will cover next.

Example 4-7. Per-key average with `reduceByKey()` and `mapValues()` in Python

```
rdd.mapValues(lambda x: (x, 1)).reduceByKey(lambda x, y: (x[0] + y[0], x[1] + y[1]))
```

Example 4-8. Per-key average with `reduceByKey()` and `mapValues()` in Scala

```
rdd.mapValues(x => (x, 1)).reduceByKey((x, y) => (x._1 + y._1, x._2 + y._2))
```

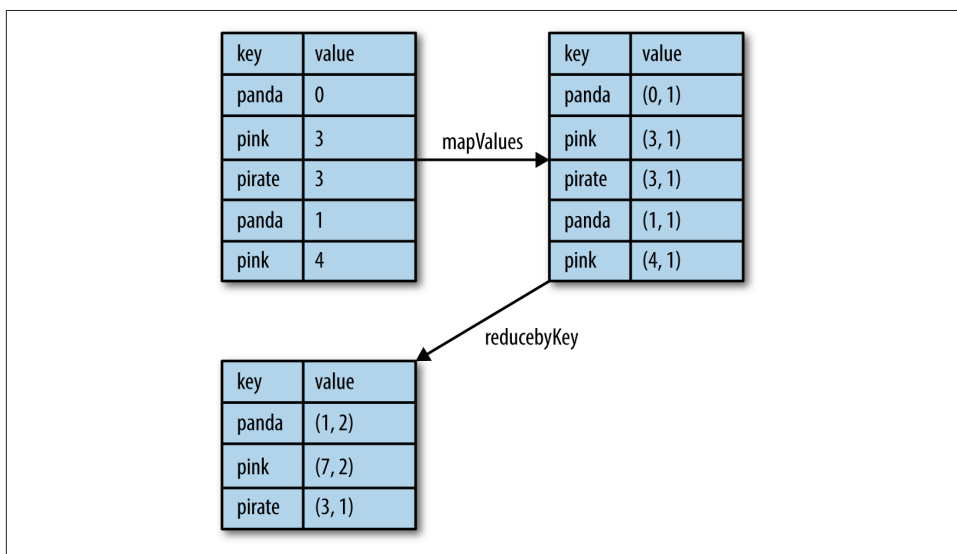


Figure 4-2. Per-key average data flow



Those familiar with the combiner concept from MapReduce should note that calling `reduceByKey()` and `foldByKey()` will automatically perform combining locally on each machine before computing global totals for each key. The user does not need to specify a combiner. The more general `combineByKey()` interface allows you to customize combining behavior.

We can use a similar approach in Examples 4-9 through 4-11 to also implement the classic distributed word count problem. We will use `flatMap()` from the previous chapter so that we can produce a pair RDD of words and the number 1 and then sum together all of the words using `reduceByKey()` as in Examples 4-7 and 4-8.

Example 4-9. Word count in Python

```
rdd = sc.textFile("s3://...")
words = rdd.flatMap(lambda x: x.split(" "))
result = words.map(lambda x: (x, 1)).reduceByKey(lambda x, y: x + y)
```

Example 4-10. Word count in Scala

```
val input = sc.textFile("s3://...")
val words = input.flatMap(x => x.split(" "))
val result = words.map(x => (x, 1)).reduceByKey((x, y) => x + y)
```

Example 4-11. Word count in Java

```
JavaRDD<String> input = sc.textFile("s3://...")
JavaRDD<String> words = rdd.flatMap(new FlatMapFunction<String, String>() {
    public Iterable<String> call(String x) { return Arrays.asList(x.split(" ")); }
});
JavaPairRDD<String, Integer> result = words.mapToPair(
    new PairFunction<String, String, Integer>() {
        public Tuple2<String, Integer> call(String x) { return new Tuple2(x, 1); }
    }).reduceByKey(
    new Function2<Integer, Integer, Integer>() {
        public Integer call(Integer a, Integer b) { return a + b; }
    });
```



We can actually implement word count even faster by using the `countByKey()` function on the first RDD: `input.flatMap(x => x.split(" ")).countByKey()`.

`combineByKey()` is the most general of the per-key aggregation functions. Most of the other per-key combinators are implemented using it. Like `aggregate()`, `combineByKey()` allows the user to return values that are not the same type as our input data.

To understand `combineByKey()`, it's useful to think of how it handles each element it processes. As `combineByKey()` goes through the elements in a partition, each element either has a key it hasn't seen before or has the same key as a previous element.

If it's a new element, `combineByKey()` uses a function we provide, called `createCombiner()`, to create the initial value for the accumulator on that key. It's important

to note that this happens the first time a key is found in each partition, rather than only the first time the key is found in the RDD.

If it is a value we have seen before while processing that partition, it will instead use the provided function, `mergeValue()`, with the current value for the accumulator for that key and the new value.

Since each partition is processed independently, we can have multiple accumulators for the same key. When we are merging the results from each partition, if two or more partitions have an accumulator for the same key we merge the accumulators using the user-supplied `mergeCombiners()` function.



We can disable map-side aggregation in `combineByKey()` if we know that our data won't benefit from it. For example, `groupByKey()` disables map-side aggregation as the aggregation function (appending to a list) does not save any space. If we want to disable map-side combines, we need to specify the partitioner; for now you can just use the partitioner on the source RDD by passing `rdd.partitioner`.

Since `combineByKey()` has a lot of different parameters it is a great candidate for an explanatory example. To better illustrate how `combineByKey()` works, we will look at computing the average value for each key, as shown in Examples 4-12 through 4-14 and illustrated in Figure 4-3.

Example 4-12. Per-key average using `combineByKey()` in Python

```
sumCount = nums.combineByKey((lambda x: (x,1)),
                             (lambda x, y: (x[0] + y, x[1] + 1)),
                             (lambda x, y: (x[0] + y[0], x[1] + y[1])))
sumCount.map(lambda key, xy: (key, xy[0]/xy[1])).collectAsMap()
```

Example 4-13. Per-key average using `combineByKey()` in Scala

```
val result = input.combineByKey(
  (v) => (v, 1),
  (acc: (Int, Int), v) => (acc._1 + v, acc._2 + 1),
  (acc1: (Int, Int), acc2: (Int, Int)) => (acc1._1 + acc2._1, acc1._2 + acc2._2)
).map{ case (key, value) => (key, value._1 / value._2.toFloat) }
result.collectAsMap().map(println(_))
```

Example 4-14. Per-key average using `combineByKey()` in Java

```
public static class AvgCount implements Serializable {
    public AvgCount(int total, int num) { total_ = total; num_ = num; }
    public int total_;
```



```

    public int num_;
    public float avg() { return total_ / (float) num_; }
}

Function<Integer, AvgCount> createAcc = new Function<Integer, AvgCount>() {
    public AvgCount call(Integer x) {
        return new AvgCount(x, 1);
    }
};

Function2<AvgCount, Integer, AvgCount> addAndCount =
    new Function2<AvgCount, Integer, AvgCount>() {
        public AvgCount call(AvgCount a, Integer x) {
            a.total_ += x;
            a.num_ += 1;
            return a;
        }
    };

Function2<AvgCount, AvgCount, AvgCount> combine =
    new Function2<AvgCount, AvgCount, AvgCount>() {
        public AvgCount call(AvgCount a, AvgCount b) {
            a.total_ += b.total_;
            a.num_ += b.num_;
            return a;
        }
    };

AvgCount initial = new AvgCount(0,0);
JavaPairRDD<String, AvgCount> avgCounts =
    nums.combineByKey(createAcc, addAndCount, combine);
Map<String, AvgCount> countMap = avgCounts.collectAsMap();
for (Entry<String, AvgCount> entry : countMap.entrySet()) {
    System.out.println(entry.getKey() + ":" + entry.getValue().avg());
}

```

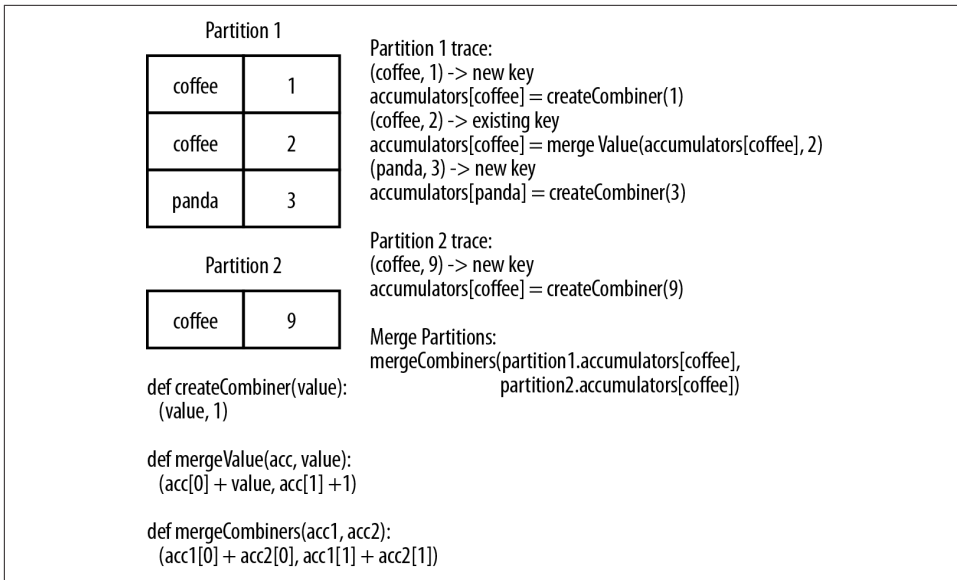


Figure 4-3. *combineByKey()* sample data flow

There are many options for combining our data by key. Most of them are implemented on top of `combineByKey()` but provide a simpler interface. In any case, using one of the specialized aggregation functions in Spark can be much faster than the naive approach of grouping our data and then reducing it.

Tuning the level of parallelism

So far we have talked about how all of our transformations are distributed, but we have not really looked at how Spark decides how to split up the work. Every RDD has a fixed number of *partitions* that determine the degree of parallelism to use when executing operations on the RDD.

When performing aggregations or grouping operations, we can ask Spark to use a specific number of partitions. Spark will always try to infer a sensible default value based on the size of your cluster, but in some cases you will want to tune the level of parallelism for better performance.

Most of the operators discussed in this chapter accept a second parameter giving the number of partitions to use when creating the grouped or aggregated RDD, as shown in Examples 4-15 and 4-16.

Example 4-15. reduceByKey() with custom parallelism in Python

```
data = [("a", 3), ("b", 4), ("a", 1)]
sc.parallelize(data).reduceByKey(lambda x, y: x + y)      # Default parallelism
sc.parallelize(data).reduceByKey(lambda x, y: x + y, 10)  # Custom parallelism
```

Example 4-16. reduceByKey() with custom parallelism in Scala

```
val data = Seq(("a", 3), ("b", 4), ("a", 1))
sc.parallelize(data).reduceByKey((x, y) => x + y)        // Default parallelism
sc.parallelize(data).reduceByKey((x, y) => x + y, 10)    // Custom parallelism
```

Sometimes, we want to change the partitioning of an RDD outside the context of grouping and aggregation operations. For those cases, Spark provides the `repartition()` function, which shuffles the data across the network to create a new set of partitions. Keep in mind that repartitioning your data is a fairly expensive operation. Spark also has an optimized version of `repartition()` called `coalesce()` that allows avoiding data movement, but only if you are decreasing the number of RDD partitions. To know whether you can safely call `coalesce()`, you can check the size of the RDD using `rdd.partitions.size()` in Java/Scala and `rdd.getNumPartitions()` in Python and make sure that you are coalescing it to fewer partitions than it currently has.

Grouping Data

With keyed data a common use case is grouping our data by key—for example, viewing all of a customer's orders together.

If our data is already keyed in the way we want, `groupByKey()` will group our data using the key in our RDD. On an RDD consisting of keys of type `K` and values of type `V`, we get back an RDD of type `[K, Iterable[V]]`.

`groupByKey()` works on unpaired data or data where we want to use a different condition besides equality on the current key. It takes a function that it applies to every element in the source RDD and uses the result to determine the key.



If you find yourself writing code where you `groupByKey()` and then use a `reduce()` or `fold()` on the values, you can probably achieve the same result more efficiently by using one of the per-key aggregation functions. Rather than reducing the RDD to an in-memory value, we reduce the data per key and get back an RDD with the reduced values corresponding to each key. For example, `rdd.reduceByKey(func)` produces the same RDD as `rdd.groupByKey().mapValues(value => value.reduce(func))` but is more efficient as it avoids the step of creating a list of values for each key.

In addition to grouping data from a single RDD, we can group data sharing the same key from multiple RDDs using a function called `cogroup()`. `cogroup()` over two RDDs sharing the same key type, `K`, with the respective value types `V` and `W` gives us back `RDD[(K, (Iterable[V], Iterable[W]))]`. If one of the RDDs doesn't have elements for a given key that is present in the other RDD, the corresponding `Iterable` is simply empty. `cogroup()` gives us the power to group data from multiple RDDs.

`cogroup()` is used as a building block for the joins we discuss in the next section.



`cogroup()` can be used for much more than just implementing joins. We can also use it to implement `intersect by key`. Additionally, `cogroup()` can work on three or more RDDs at once.

Joins

Some of the most useful operations we get with keyed data comes from using it together with other keyed data. Joining data together is probably one of the most common operations on a pair RDD, and we have a full range of options including right and left outer joins, cross joins, and inner joins.

The simple `join` operator is an inner join.¹ Only keys that are present in both pair RDDs are output. When there are multiple values for the same key in one of the inputs, the resulting pair RDD will have an entry for every possible pair of values with that key from the two input RDDs. A simple way to understand this is by looking at [Example 4-17](#).

Example 4-17. Scala shell inner join

```
storeAddress = {
  (Store("Ritual"), "1026 Valencia St"), (Store("Philz"), "748 Van Ness Ave"),
  (Store("Philz"), "3101 24th St"), (Store("Starbucks"), "Seattle")}

storeRating = {
  (Store("Ritual"), 4.9), (Store("Philz"), 4.8)}

storeAddress.join(storeRating) == {
  (Store("Ritual"), ("1026 Valencia St", 4.9)),
  (Store("Philz"), ("748 Van Ness Ave", 4.8)),
  (Store("Philz"), ("3101 24th St", 4.8))}
```

¹ "Join" is a database term for combining fields from two tables using common values.

Sometimes we don't need the key to be present in both RDDs to want it in our result. For example, if we were joining customer information with recommendations we might not want to drop customers if there were not any recommendations yet. `leftOuterJoin(other)` and `rightOuterJoin(other)` both join pair RDDs together by key, where one of the pair RDDs can be missing the key.

With `leftOuterJoin()` the resulting pair RDD has entries for each key in the source RDD. The value associated with each key in the result is a tuple of the value from the source RDD and an `Option` (or `Optional` in Java) for the value from the other pair RDD. In Python, if a value isn't present `None` is used; and if the value is present the regular value, without any wrapper, is used. As with `join()`, we can have multiple entries for each key; when this occurs, we get the Cartesian product between the two lists of values.



`Optional` is part of [Google's Guava library](#) and represents a possibly missing value. We can check `isPresent()` to see if it's set, and `get()` will return the contained instance provided data is present.

`rightOuterJoin()` is almost identical to `leftOuterJoin()` except the key must be present in the other RDD and the tuple has an option for the source rather than the other RDD.

We can revisit [Example 4-17](#) and do a `leftOuterJoin()` and a `rightOuterJoin()` between the two pair RDDs we used to illustrate `join()` in [Example 4-18](#).

Example 4-18. `leftOuterJoin()` and `rightOuterJoin()`

```
storeAddress.leftOuterJoin(storeRating) ==
{(Store("Ritual"),("1026 Valencia St",Some(4.9))),
 (Store("Starbucks"),("Seattle",None)),
 (Store("Philz"),("748 Van Ness Ave",Some(4.8))),
 (Store("Philz"),("3101 24th St",Some(4.8)))}

storeAddress.rightOuterJoin(storeRating) ==
{(Store("Ritual"),(Some("1026 Valencia St"),4.9)),
 (Store("Philz"),(Some("748 Van Ness Ave"),4.8)),
 (Store("Philz"), (Some("3101 24th St"),4.8))}
```

Sorting Data

Having sorted data is quite useful in many cases, especially when you're producing downstream output. We can sort an RDD with key/value pairs provided that there is an ordering defined on the key. Once we have sorted our data, any subsequent call on the sorted data to `collect()` or `save()` will result in ordered data.